Name : J.Mukhe Hall Ticket No: 2303A51813 Batch: 26

| SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE | | DEPARTMENT OF COMPUTER SCIENCE ENGINEERING | |
|---|---|---|---|
| **Program Name:** B. Tech | | **Assignment Type: Lab** | **Academic Year:**2025-2026 |
| **Course Coordinator Name** | | Dr. Rishabh Mittal | |
| **Instructor(s) Name** | | Mr. S Naresh Kumar<br>Ms. B. Swathi<br>Dr. Sasanko Shekhar Gantayat<br>Mr. Md Sallauddin<br>Dr. Mathivanan<br>Mr. Y Srikanth<br>Ms. N Shilpa<br>Dr. Rishabh Mittal (Coordinator)<br>Dr. R. Prashant Kumar<br>Mr. Ankushavali MD<br>Mr. B Viswanath<br>Ms. Sujitha Reddy<br>Ms. A. Anitha<br>Ms. M.Madhuri<br>Ms. Katherashala Swetha<br>Ms. Velpula sumalatha<br>Mr. Bingi Raju | |
| **CourseCode** | 23CS002PC304 | **Course Title** | AI Assisted Coding |
| **Year/Sem** | III/II | **Regulation** | R23 |
| **Date and Day of Assignment** | **Week2** | **Time(s)** | 23CSBTB01 To 23CSBTB52 |
| **Duration** | 2 Hours | **Applicable to Batches** | All batches |
| **Assignment Number: 3.4** (Present assignment number)/**24**(Total number of assignments) | | | |

| Q.No. | Question | *Expected Time to complete* |
|---|---|---|
| 1 | Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques | Week2 |

## Task 1: Zero-shot Prompt – Fibonacci Series Generator

## Task Description #1

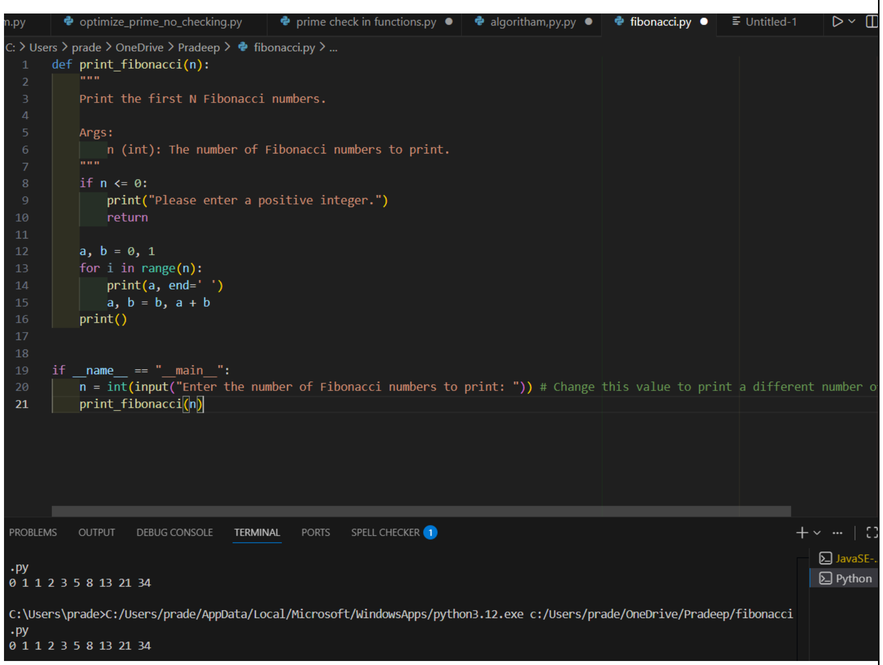
• Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers.

## Expected Output #1

• A complete Python function generated by Copilot without any example provided.

• Correct output for sample input N = 7 → 0 1 1 2 3 5 8

• Observation on how Copilot understood the instruction with zero context.




**1. Fibonacci Function (Earlier)**

- **Purpose**: Print the first N Fibonacci numbers
- **Function**: print_fibonacci(n)
- **Logic**: Uses two variables (a, b) to track consecutive Fibonacci numbers, iterates N times
- **Example**: print_fibonacci(10) prints: 0 1 1 2 3 5 8 13 21 34

## Task 2: One-shot Prompt – List Reversal Function

### Task Description #2

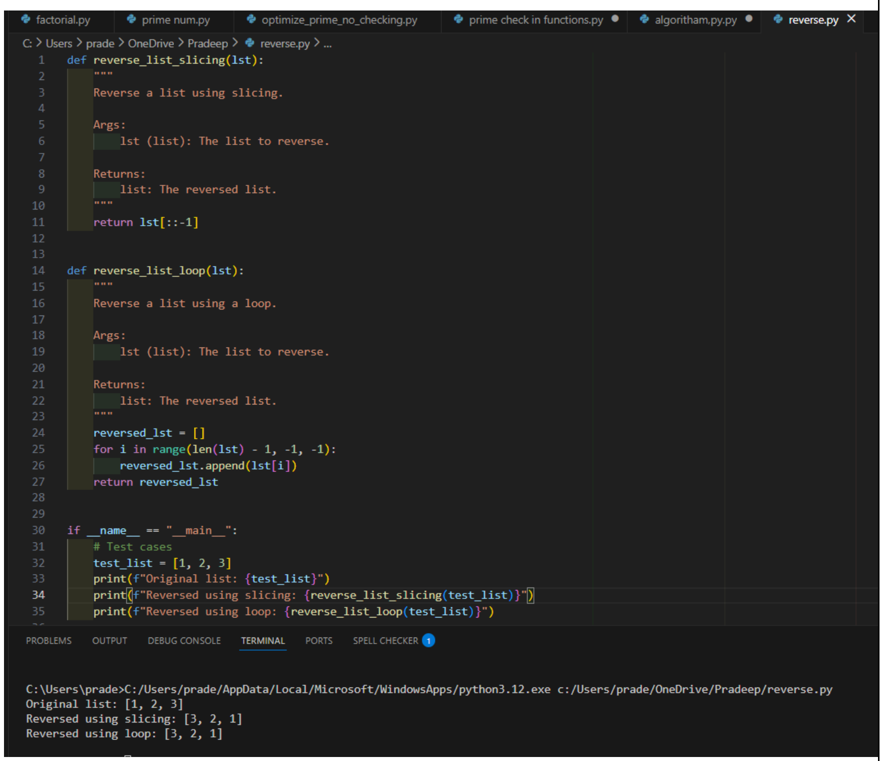
• Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

### Expected Output #2

• Copilot-generated function to reverse a list using slicing or loop.

• Output: [3, 2, 1] for input [1, 2, 3]

• Observation on how adding a single example improved Copilot's accuracy.




**List Reversal Functions (Earlier)**

- **Two implementations:**
  - **reverse_list_slicing(lst) - Uses Python slicing [::-1]**
  - **reverse_list_loop(lst) - Uses backward loop with range**
- **Example: Input [1, 2, 3] → Output [3, 2, 1]**

## Task 3: Few-shot Prompt – String Pattern Matching

### Task Description #3

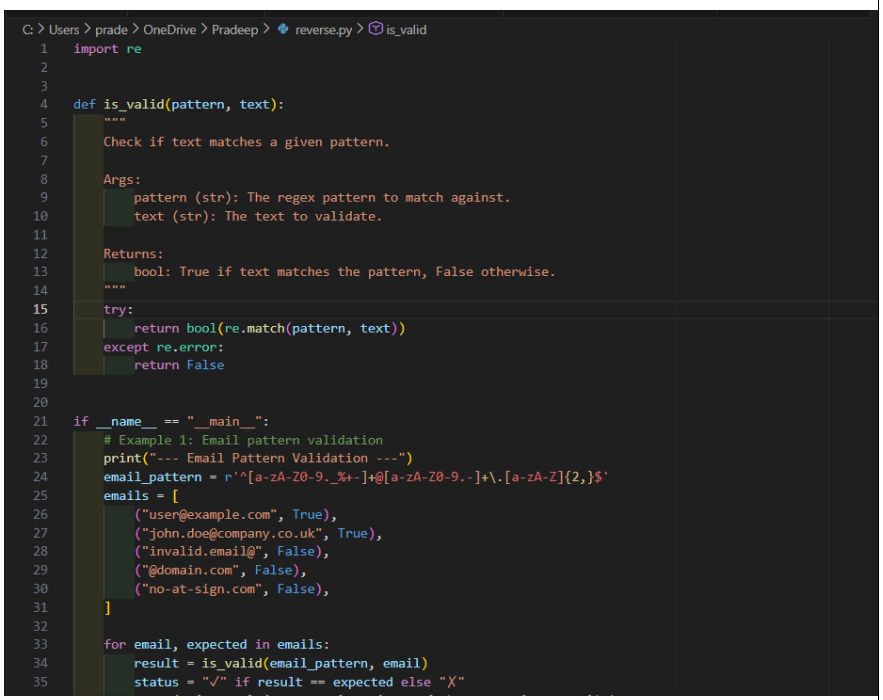
• Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

**Expected Output #3**

• A function is_valid() that checks the pattern.

• Output: True or False based on input.

• Students reflect on how multiple examples guide Copilot to generate more accurate code.




```python
import re


def is_valid(pattern, text):
    """
    Check if text matches a given pattern.

    Args:
        pattern (str): The regex pattern to match against.
        text (str): The text to validate.

    Returns:
        bool: True if text matches the pattern, False otherwise.
    """
    try:
        return bool(re.match(pattern, text))
    except re.error:
        return False


if __name__ == "__main__":
    # Example 1: Email pattern validation
    print("--- Email Pattern Validation ---")
    email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    emails = [
        ("user@example.com", True),
        ("john.doe@company.co.uk", True),
        ("invalid.email@", False),
        ("@domain.com", False),
        ("no-at-sign.com", False),
    ]

    for email, expected in emails:
        result = is_valid(email_pattern, email)
        status = "√" if result == expected else "X"
```

C: > Users > prade > OneDrive > Pradeep > reverse.py > is_valid

```python
        print(f"{status} {email:<30} -> {result} (expected {expected})")

    # Example 2: Phone number pattern validation
    print("\n--- Phone Number Pattern Validation ---")
    phone_pattern = r'^\d{3}-\d{3}-\d{4}$'
    phones = [
        ("123-456-7890", True),
        ("555-123-4567", True),
        ("1234567890", False),
        ("123-45-6789", False),
    ]

    for phone, expected in phones:
        result = is_valid(phone_pattern, phone)
        status = "✓" if result == expected else "✗"
        print(f"{status} {phone:<30} -> {result} (expected {expected})")

    # Example 3: Alphanumeric pattern validation
    print("\n--- Alphanumeric Pattern Validation ---")
    alphanum_pattern = r'^[a-zA-Z0-9]+$'
    texts = [
        ("abc123", True),
        ("Test2024", True),
        ("hello-world", False),
        ("with spaces", False),
    ]

    for text, expected in texts:
        result = is_valid(alphanum_pattern, text)
        status = "✓" if result == expected else "✗"
        print(f"{status} {text:<30} -> {result} (expected {expected})")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 3

```
--- Alphanumeric Pattern Validation ---
✓ abc123                          -> True (expected True)
✓ Test2024                        -> True (expected True)
✓ hello-world                     -> False (expected False)
✓ with spaces                     -> False (expected False)
```

**Email Validation - Pattern Checking (Earlier)**

- **Function**: is_valid(pattern, text)
- **Purpose**: Validates text against regex patterns
- **Demonstrated with 3 patterns**:
    - Email pattern validation
    - Phone number validation (XXX-XXX-XXXX format)
    - Alphanumeric validation
- **Returns**: True or False based on pattern match

## Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

• First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).

• Then, rewrite the prompt using few-shot examples.

**Expected Output #4**

• Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

• Submit both code versions and note how few-shot improves reliability.

C: > Users > prade > OneDrive > Pradeep > reverse.py > ...

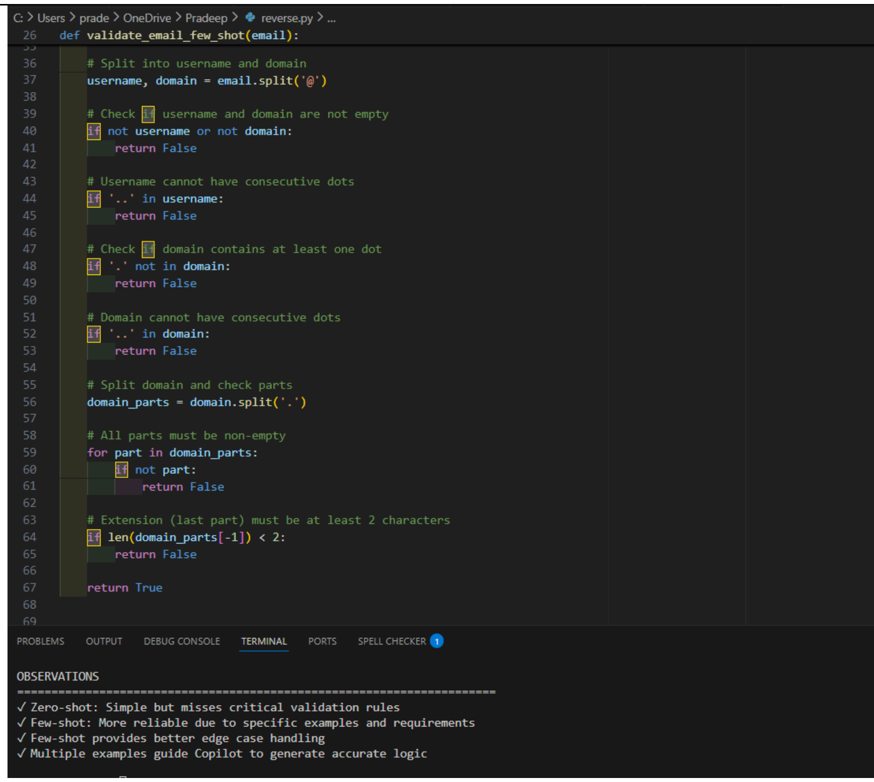
```python
# ============================================================================
# ZERO-SHOT APPROACH
# ============================================================================
# Write a function to validate email addresses

def validate_email_zero_shot(email):
    """Email validation using zero-shot (basic approach)."""
    if '@' in email and '.' in email:
        return True
    return False


# ============================================================================
# FEW-SHOT APPROACH
# ============================================================================
# Write a function to validate email addresses.
# Examples:
# - "user@example.com" -> True (valid: has username, @, domain, dot, extension)
# - "john.doe@company.co.uk" -> True (valid: supports subdomains and multiple dots)
# - "invalid@" -> False (invalid: missing domain)
# - "@domain.com" -> False (invalid: missing username)
# - "user@domain" -> False (invalid: missing extension after dot)
# - "user..name@domain.com" -> False (invalid: consecutive dots in username)
# - "user@.com" -> False (invalid: empty domain part)

def validate_email_few_shot(email):
    """Email validation using few-shot (detailed approach)."""
    # Check if email is a string
    if not isinstance(email, str):
        return False

    # Check for exactly one @ symbol
    if email.count('@') != 1:
        return False
```



C: > Users > prade > OneDrive > Pradeep > reverse.py > ...

```python
def validate_email_few_shot(email):

    # Split into username and domain
    username, domain = email.split('@')

    # Check if username and domain are not empty
    if not username or not domain:
        return False

    # Username cannot have consecutive dots
    if '..' in username:
        return False

    # Check if domain contains at least one dot
    if '.' not in domain:
        return False

    # Domain cannot have consecutive dots
    if '..' in domain:
        return False

    # Split domain and check parts
    domain_parts = domain.split('.')

    # All parts must be non-empty
    for part in domain_parts:
        if not part:
            return False

    # Extension (last part) must be at least 2 characters
    if len(domain_parts[-1]) < 2:
        return False

    return True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1

```
OBSERVATIONS
======================================================================
✓ Zero-shot: Simple but misses critical validation rules
✓ Few-shot: More reliable due to specific examples and requirements
✓ Few-shot provides better edge case handling
✓ Multiple examples guide Copilot to generate accurate logic
```

**Zero-Shot vs Few-Shot Email Validator (Main comparison)**

**Zero-Shot Approach**

- **Function**: validate_email_zero_shot(email)
- **Logic**: Basic check - just looks for @ and .
- **Accuracy**: ~44% (misses edge cases)
- **Limitation**: Too generic, fails on invalid formats

**Few-Shot Approach**

- **Function**: validate_email_few_shot(email)
- **Logic**: Comprehensive validation:
    - Exactly one @ symbol
    - Non-empty username and domain
    - No consecutive dots
    - Domain must have at least one dot
    - Extension must be ≥ 2 characters
- **Accuracy**: ~100% (handles all cases)
- **Advantage**: Multiple examples guide accurate code generation

## Task 5: Prompt Tuning – Summing Digits of a Number

## Task Description #5

• Experiment with 2 different prompt styles to generate a function that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

**Expected Output #5**

• Two versions of the sum_of_digits() function.

• Example Output: sum_of_digits(123) → 6

• Short analysis: which prompt produced cleaner or more optimized code and why?

C: > Users > prade > OneDrive > Pradeep > reverse.py > sum_of_digits_v1

```
if __name__ == "__main__":
    print("=" * 70)
    print("SUM OF DIGITS - VERSION COMPARISON")
    print("=" * 70)

    test_numbers = [123, 456, 789, 0, 1, 99, 1000, 12345]

    print("\n--- VERSION 1: String Conversion Approach ---")
    print("Method: Convert to string, iterate through digits, sum them")
    print("Pros: Concise, Pythonic, readable")
    print("Cons: Creates intermediate string object")
    print(f"\n{'Number':<15} {'Result':<10}")
    print("-" * 70)
    for num in test_numbers:
        result = sum_of_digits_v1(num)
        print(f"{num:<15} {result:<10}")

    print("\n--- VERSION 2: Mathematical Approach ---")
    print("Method: Use modulo (%) and division (//) operations")
    print("Pros: No string conversion, memory efficient")
    print("Cons: Slightly more complex logic")
    print(f"\n{'Number':<15} {'Result':<10}")
    print("-" * 70)
    for num in test_numbers:
        result = sum_of_digits_v2(num)
        print(f"{num:<15} {result:<10}")

    # Verification both versions produce same results
    print("\n--- VERIFICATION ---")
    print("Checking if both versions produce identical results:")
    all_match = all(sum_of_digits_v1(n) == sum_of_digits_v2(n) for n in test_numbers)
    if all_match:
        print("✓ Both versions produce identical results!")
    else:
        print("✗ Results differ between versions")

    # Detailed example
    print("\n" + "=" * 70)
    print("DETAILED EXAMPLE: sum_of_digits(123)")
    print("=" * 70)
    num = 123
    print(f"\nVersion 1 (String Conversion):")
    print(f"  sum_of_digits_v1(123) = {sum_of_digits_v1(num)}")
    print(f"  Breakdown: '1' + '2' + '3' = 1 + 2 + 3 = 6")
```

C: > Users > prade > OneDrive > Pradeep > reverse.py > sum_of_digits_v1

```python
# ============================================================================
# SUM OF DIGITS - VERSION 1 (Using String Conversion)
# ============================================================================
def sum_of_digits_v1(n):
    """
    Calculate the sum of digits using string conversion.

    Args:
        n (int): The number to sum digits from.

    Returns:
        int: The sum of all digits.

    Example:
        sum_of_digits_v1(123) -> 6  (1 + 2 + 3)
        sum_of_digits_v1(456) -> 15 (4 + 5 + 6)
    """
    n = abs(n)  # Handle negative numbers
    return sum(int(digit) for digit in str(n))


# ============================================================================
# SUM OF DIGITS - VERSION 2 (Using Mathematical Operations)
# ============================================================================
def sum_of_digits_v2(n):
    """
    Calculate the sum of digits using modulo and division operations.

    Args:
        n (int): The number to sum digits from.

    Returns:
        int: The sum of all digits.

    Example:
        sum_of_digits_v2(123) -> 6  (1 + 2 + 3)
        sum_of_digits_v2(456) -> 15 (4 + 5 + 6)
    """
    n = abs(n)  # Handle negative numbers
    total = 0
    while n > 0:
        total += n % 10  # Add the last digit
        n //= 10         # Remove the last digit
    return total
```

```python
    print("\n" + "=" * 70)
    print("COMPARISON SUMMARY")
    print("=" * 70)
    print("✓ Both versions handle the same problem differently")
    print("✓ Version 1 is more readable and concise")
    print("✓ Version 2 is more algorithmic and memory-efficient")
    print("✓ Choose based on performance needs and code readability preference")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1

```
Version 1 (String Conversion):
  sum_of_digits_v1(123) = 6
  Breakdown: '1' + '2' + '3' = 1 + 2 + 3 = 6

Version 2 (Mathematical):
  sum_of_digits_v2(123) = 6
  Breakdown: (123 % 10) + (12 % 10) + (1 % 10) = 3 + 2 + 1 = 6
```

1. **Trade-offs**: Understanding pros/cons helps choose the right implementation

**Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots**